Dynamic Device Management Made Easy with udev

Connecting flash drives, hard disks, cameras or mobile phones to your Linux system has never been so easy and manageable, thanks to udev, developed by Greg Kroah-Hartman, Kay Sievers and Dan Stekloff. First implemented in Linux kernel 2.6, udev handles devices that are hot-plugged into a running system, as well as cold-plugged devices (connected before the system is powered on). In this article, we cover how udev dynamically adds device nodes to the /dev directory, and provide some examples of configurations for your use or amusement.

Udev stands for 'userspace implementation of devfs'. It includes a *udevd* daemon, configuration files and rule files, which are used to dynamically manage device files in the */dev/* directory in Linux, in response to *uevents* generated by the kernel. udev has successfully and completely replaced the older *devfs* since the Linux kernel 2.6 series.

Why did we need a totally new implementation of device mapping? Why has udev been such a success? The answer involves the history of the Linux device driver mapping scheme.

Each device file is assigned two 8-bit numbers: a major number and a minor number. Each device driver has a major device number; and all device files for devices controlled by that driver have the same major number. Minor device numbers distinguish between different devices controlled by the driver.

In earlier Linux kernel versions, */dev* contained one static device file for each device that could be connected to the system (and controlled by a device driver). Unfortunately, this had problems: there weren't sufficient numbers to handle all the possibilities,